Metody Zespołowe (Ensemble Learning)

Lasy Losowe, Gradient Boosting, AdaBoost

Szymon Zaporowski

Kierunek: Informatyka, II stopień, sem. 1
Metody Sztucznej Inteligencji

24 kwietnia 2026

Plan wykładu

1. Motywacja i taksonomia
2. Drzewa decyzyjne – fundament
3. Lasy Losowe i Lasy Regresyjne
4. AdaBoost
5. Gradient Boosting
6. Porównanie i implementacja

Plan wykładu

1. Motywacja i taksonomia
2. Drzewa decyzyjne – fundament
3. Lasy Losowe i Lasy Regresyjne
4. AdaBoost
5. Gradient Boosting
6. Porównanie i implementacja

Dlaczego metody zespołowe?

Problem pojedynczego modelu:

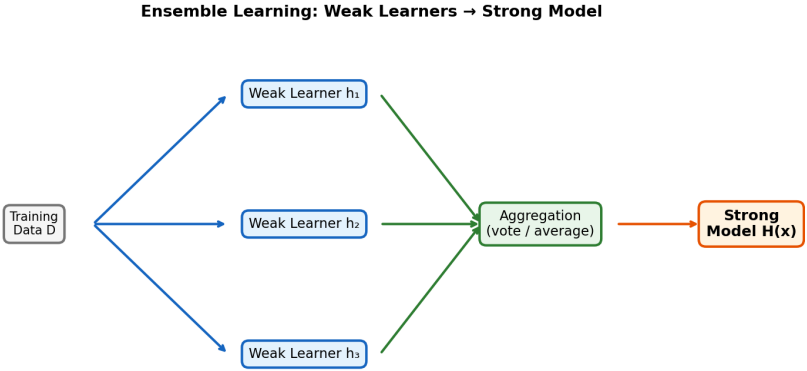
- Wysokie odchylenie (*bias*) – niedotrenowanie
- Wysoka wariancja (*variance*) – przetrenowanie
- Kompromis bias–variance jest trudny do osiągnięcia

Idea: *„Mądrość tłumu”* – wiele słabych modeli może tworzyć jeden silny.

Uczenie zespołowe – definicja

Definicja

Uczenie zespołowe (*Ensemble Learning*) to technika łączenia wielu modeli w celu uzyskania lepszej predykcji niż możliwa przez pojedynczy algorytm.



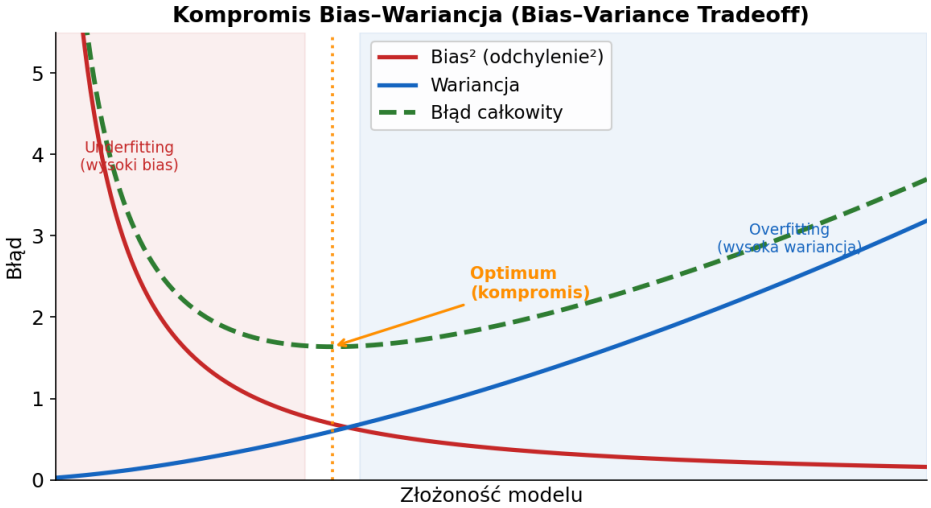
Kompromis Bias–Wariancja

Rozkład błędu całkowitego:

$$\mathbb{E}[(y - \hat{f})^2] = \underbrace{\mathrm{Bias}^2(\hat{f})}_{\text{niedouczenie}} + \underbrace{\mathrm{Var}(\hat{f})}_{\text{przeuczenie}} + \sigma_\varepsilon^2$$

- **Bagging** (RF): redukuje **wariancję** przez uśrednianie nieskorelowanych modeli
- **Boosting**: redukuje **odchylenie** przez iteracyjne korygowanie błędów
- σ_ε^2 – nieredukowalny szum

Kompromis Bias–Wariancja – ilustracja



Krzywa błędu treningowego i testowego w funkcji złożoności modelu

Taksonomia metod zespołowych

Metody równoległe (Bagging)

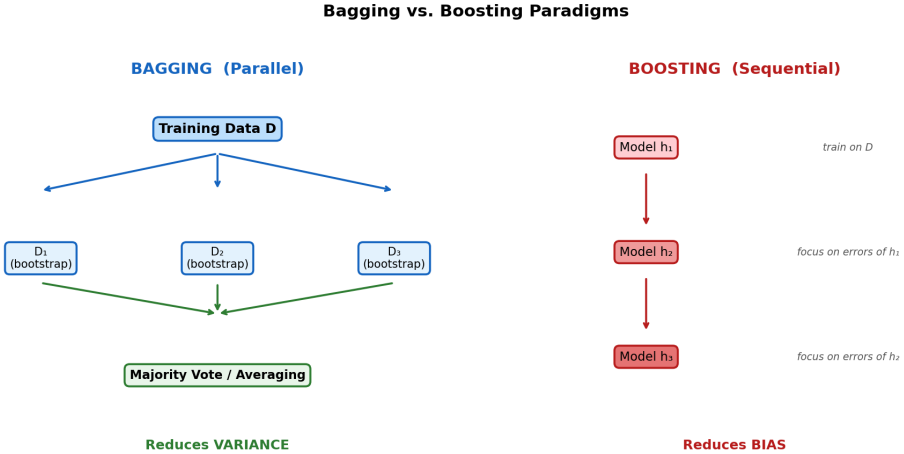
- Modele trenowane **niezależnie**
- Agregacja: uśrednianie / głosowanie
- Cel: redukcja **wariancji**
- Przykład: **Random Forest**

Metody sekwencyjne (Boosting)

- Modele trenowane **iteracyjnie**
- Każdy model uczy się na błędach poprzedniego
- Cel: redukcja **odchylenia**
- Przykłady: **AdaBoost**, **GBM**

Trzecia droga – **Stacking** – sekcja 6.

Bagging vs. Boosting – schemat działania



Źródło: wygenerowano w Python

Plan wykładu

1. Motywacja i taksonomia
2. Drzewa decyzyjne – fundament
3. Lasy Losowe i Lasy Regresyjne
4. AdaBoost
5. Gradient Boosting
6. Porównanie i implementacja

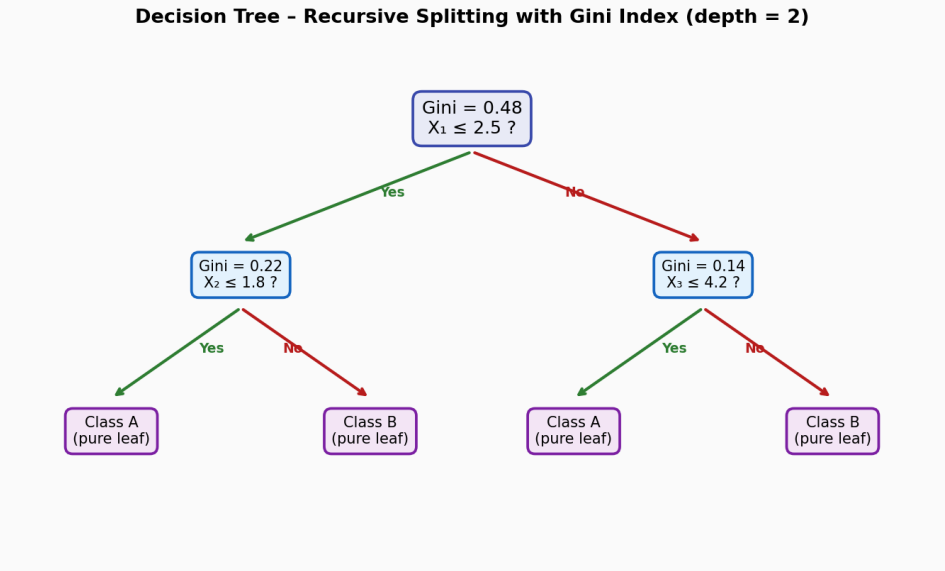
Drzewa decyzyjne – szybkie przypomnienie

- Rekurencyjny podział przestrzeni cech
- Węzły = warunki na cechach; Liście = klasa / wartość
- Kryterium podziału – klasyfikacja: **indeks Giniego**

$$G = 1 - \sum_k p_k^2$$

- Kryterium podziału – regresja: **MSE**

Drzewa decyzyjne – problem wariancji



Plan wykładu

1. Motywacja i taksonomia
2. Drzewa decyzyjne – fundament
3. Lasy Losowe i Lasy Regresyjne
4. AdaBoost
5. Gradient Boosting
6. Porównanie i implementacja

Bootstrap Sampling – intuicja

Losowanie ze zwracaniem z $\mathcal{D}$ o n próbkach:

- Każde $\mathcal{D}_t$ ma n elementów (możliwe duplikaty)
- Średnia frakcja **unikalnych** próbek w $\mathcal{D}_t$:

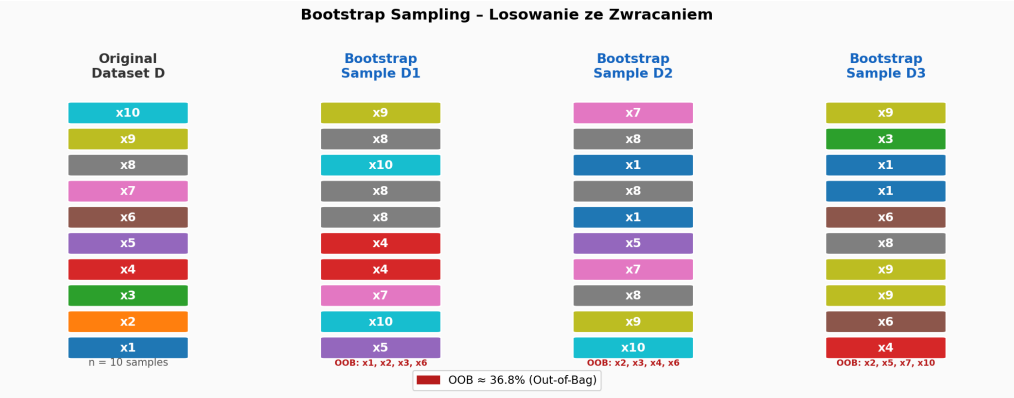
$$1 - \lim_{n\to\infty}\left(1 - \frac{1}{n}\right)^n = 1 - \frac{1}{e} \approx \mathbf{63{,}2\%}$$

- Pozostałe $\approx$ **36,8% – Out-of-Bag (OOB)**
- OOB = darmowy zbiór walidacyjny bez walidacji krzyżowej!

Dlaczego to ważne?

OOB error $\approx$ nieobciążony estymator błędu generalizacji.

Bootstrap Sampling – ilustracja



Każde $\mathcal{D}_t$ zawiera inne (z powt.) próbki; próbki **OOB** (≈36,8%) służą jako zbiór walidacyjny.

Las Losowy – algorytm

1. **Bootstrap sampling:** dla każdego z T drzew wylosuj próbę $\mathcal{D}_t \subset \mathcal{D}$ (ze zwracaniem, $|\mathcal{D}_t| = n$)
2. **Random subspace:** przy każdym węźle wybierz losowo m cech

$$m = \begin{cases} \lfloor \sqrt{p} \rfloor & \text{(klasyfikacja)} \\ \lfloor p/3 \rfloor & \text{(regresja)} \end{cases}$$

3. **Buduj drzewo** h_t – domyślnie do pełnej głębokości
4. **Agregacja:**
   - Klasyfikacja: $\hat{y} = \text{majority_vote}(h_1, \ldots, h_T)$
   - Regresja: $\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$

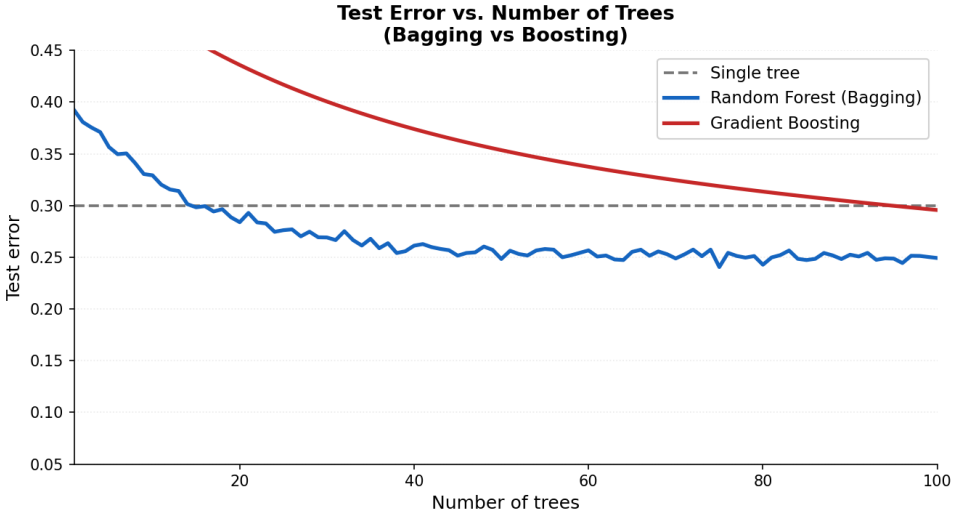
Kluczowa obserwacja

Dekorelacja drzew przez losowy dobór cech obniża wariancję przy zachowaniu niskiego odchylenia. Regularyzacja: `max_depth`, `min_samples_leaf`.

Lasy Losowe – właściwości

- **OOB error:** darmowa estymacja błędu generalizacji (każda próbka OOB dla $\sim$37% drzew)
- **Feature Importance (MDI):** suma poprawy Gini po wszystkich podziałach używających cechy j
- **Uwaga:** MDI preferuje cechy o wielu unikalnych wartościach $\Rightarrow$ alternatywa: **Permutation Importance**
- Odporność na szum i wartości odstające
- **RF nie przeuczy się** przy rosnącej liczbie drzew – błąd generalizacji zbiega asymptotycznie (Breiman 2001)

Lasy Losowe – zbieżność błędu



Plan wykładu

1. Motywacja i taksonomia
2. Drzewa decyzyjne – fundament
3. Lasy Losowe i Lasy Regresyjne
4. AdaBoost
5. Gradient Boosting
6. Porównanie i implementacja

AdaBoost – idea

Freund & Schapire, 1995

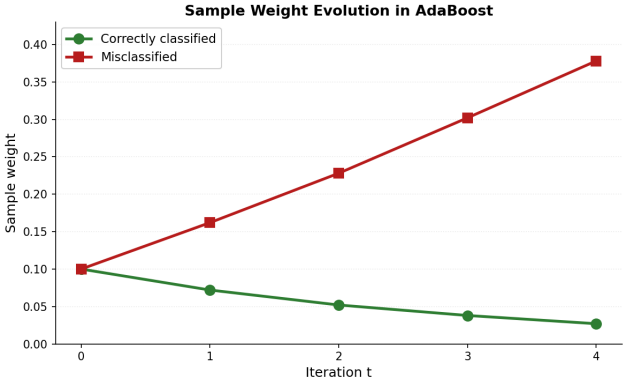
Adaptive Boosting – każdy kolejny słaby klasyfikator skupia się na próbkach, które poprzedni źle sklasyfikował.

Słaby klasyfikator (*weak learner*): model lepszy niż losowe zgadywanie ($\epsilon < 0.5$); najczęściej **decision stump** (głębokość 1).

Mechanizm:

- Próbki mają wagi w_i
- Błędnie sklasyfikowane $\Rightarrow$ większa waga
- Klasyfikatory: wagi $\alpha_t \propto$ dokładność

AdaBoost – ewolucja wag próbek



Wagi próbek **błędnie** sklasyfikowanych rosną w kolejnych iteracjach $\Rightarrow$ każdy następny klasyfikator skupia się na *trudniejszych* przypadkach.

AdaBoost – algorytm

Dane: $(x_i, y_i)_{i=1}^n, \quad y_i \in \{-1, +1\}$

1. Inicjalizacja: $w_i^{(1)} = \frac{1}{n}$
2. Dla $t = 1, \ldots, T$:
   a. Trenuj h_t na wagach $w^{(t)}$
   b. Błąd ważony: $\epsilon_t = \sum_{i:\, h_t(x_i) \neq y_i} w_i^{(t)}$
   c. Waga klasyfikatora: $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$
   d. Aktualizacja wag: $w_i^{(t+1)} \propto w_i^{(t)} \exp\big(-\alpha_t\, y_i\, h_t(x_i)\big)$
3. Predykcja: $H(x) = \operatorname{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$

Interpretacja: AdaBoost minimalizuje $L = \sum_i \exp(-y_i F(x_i))$. Wada: wrażliwość na szum i outliery.

Plan wykładu

1. Motywacja i taksonomia
2. Drzewa decyzyjne – fundament
3. Lasy Losowe i Lasy Regresyjne
4. AdaBoost
5. Gradient Boosting
6. Porównanie i implementacja

Gradient Boosting – idea i pseudoreszty

Friedman, 2001

Budowanie modelu = **optymalizacja w przestrzeni funkcji**. Każdy kolejny model aproksymuje ujemny gradient funkcji straty.

Pseudoreszty – nowe etykiety w iteracji m:

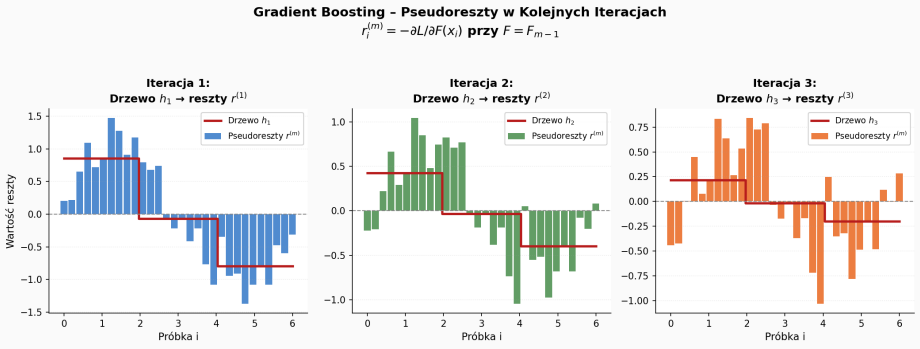
$$r_i^{(m)} = -\left.\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right|_{F=F_{m-1}}$$

- MSE: $r_i = y_i - F_{m-1}(x_i)$
- Log-loss: $r_i = y_i - p_i$

GB vs AdaBoost

AdaBoost zmienia **wagi próbek**; Gradient Boosting zmienia **etykiety** (pseudoreszty).

Gradient Boosting – sekwencja korekcji



Każde kolejne drzewo dopasowuje się do **pseudoreszt** – różnicy między aktualną predykcją $F_{m-1}(x)$ a prawdziwą wartością y.

Gradient Boosting – algorytm

1. Inicjalizacja: $F_0(x) = \arg\min_\gamma \sum_i L(y_i, \gamma)$
2. Dla $m = 1, \ldots, M$:
   a. Pseudoreszty: $r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$
   b. Dopasuj drzewo h_m do (x_i, r_{im})
   c. Optymalny krok: $\gamma_m = \arg\min_\gamma \sum_i L\big(y_i,\ F_{m-1}(x_i) + \gamma\, h_m(x_i)\big)$
   d. Aktualizacja: $F_m(x) = F_{m-1}(x) + \nu \cdot \gamma_m \cdot h_m(x)$

Hiperparametry: $\nu \in (0, 1]$ (learning rate – małe ν = lepsza generalizacja, więcej drzew); `max_depth` (typowo 3–8); `subsample`<1 (Stochastic GB, Friedman).

Nowoczesne warianty Gradient Boosting

Biblioteka	Rok	Kluczowa innowacja	Kiedy wybrać
XGBoost	2016	Regularyzacja L1/L2; obsługa brak. danych; równolegle budowanie drzew	Kaggle, średnie zbiory
LightGBM	2017	Histogram-based splitting; leaf-wise growth; mniejsze RAM	Duże zbiory (>1M rekordów), szybki prototyping
CatBoost	2018	Natywne cechy kategoryczne; symmetric trees; ordered boost.	Dane z dużym udziałem zmiennych kategorycznych

Uogólnienie

AdaBoost jest szczególnym przypadkiem Gradient Boosting z **eksponencjalną funkcją straty**.

Ke et al. (2017), NeurIPS – LightGBM paper

Plan wykładu

1. Motywacja i taksonomia
2. Drzewa decyzyjne – fundament
3. Lasy Losowe i Lasy Regresyjne
4. AdaBoost
5. Gradient Boosting
6. Porównanie i implementacja

Stacking i Voting – inne metody zespołowe

Voting Classifier

- **Hard voting:** klasa z największą liczbą głosów
- **Soft voting:** średnia prawdopodobieństw klas
- Modele mogą być **różnych typów** (SVM + RF + LR)
- W sklearn: `VotingClassifier`

Stacking (Wolpert 1992)

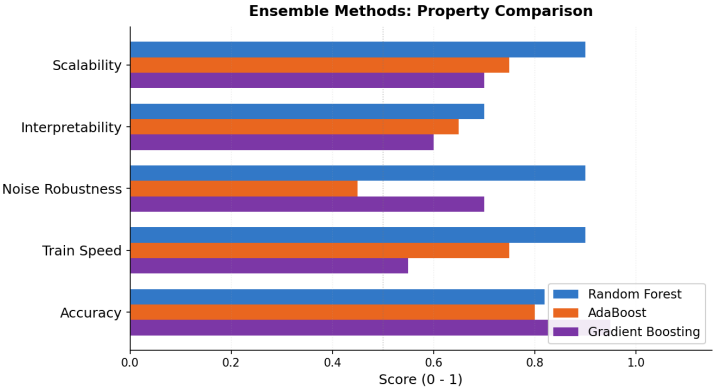
- Modele L0 (*base learners*) trenowane na danych
- **Meta-model** (L1) uczy się jak łączyć predykcje
- **Uwaga:** walidacja krzyżowa na L0 obowiązkowy (zapobieganie wyciekowi danych!)
- W sklearn: `StackingClassifier`

Porównanie metod – zestawienie

Cecha	**RF**	**AdaBoost**	**GBM**
Paradygmat	Bagging	Boosting	Boosting
Funkcja straty	impl.	exp.	dowolna
Czas treningu	●●●	●●	●
Odp. na szum	●●●	●	●●
Interpretowalność	●●	●●	●●
Maks. dokładność	●●	●●	●●●

● = niska/wolna/słaba ●● = średnia ●●● = wysoka/szybka/dobra

Porównanie metod – wizualizacja



GBM dominuje pod względem dokładności; **RF** wyróżnia się odpornością na szum i szybkością treningu.

Zaawansowane metody zespołowe (1/2)

Snapshot Ensembles

Huang et al., ICLR 2017

- Jedna sesja treningowa $\Rightarrow$ M modeli przez cykliczny *cosine annealing*:
  $\eta_t = \frac{\eta_0}{2}\left(1 + \cos\frac{\pi\,(t \bmod \lceil T/M \rceil)}{\lceil T/M \rceil}\right)$
- Snapshot zapisywany przy każdym lokalnym minimum – **koszt = trening 1 sieci**

Stochastic Weight Averaging (SWA)

Izmailov et al., UAI 2018

- Uśrednianie **wag** (nie predykcji) wzdłuż trajektorii SGD: $\bar{w} = \frac{1}{K}\sum_{k=1}^{K} w_k$
- Szeroki, płaskie minimum $\Rightarrow$ lepsza generalizacja; `torch.optim.swa_utils`

Wspólna idea

Oba podejścia uzyskują efekt ensemble **bez dodatkowego kosztu** treningu – eksplorują różne regiony przestrzeni parametrów jednego modelu.

Zaawansowane metody zespołowe (2/2)

Mixture of Experts (MoE)

Jacobs 1991; Sparse MoE – Shazeer 2017

- K wyspecjalizowanych ekspertów + sieć bramkująca $G(x)$: $\hat{y} = \sum_{k=1}^{K} G_k(x)\, f_k(x)$
- **Sparse MoE:** Top-r routing – aktywuje tylko $r \ll K$ ekspertów
- Fundament nowoczesnych LLM: Mixtral, GPT-4

Deep Ensembles

Lakshminarayanan et al., NeurIPS 2017

- M sieci z **różnymi inicjalizacjami** $\Rightarrow$ różne minima funkcji straty
- Estymacja niepewności epistemicznej: $\widehat{\mathrm{Var}}[\hat{y}] \approx \frac{1}{M}\sum_m \hat{y}_m^2 - \bar{\hat{y}}^{\,2}$
- Złoty standard kalibracji – przewyższa MC-Dropout i przybliżone BNN

Wspólna idea

Oba podejścia modelują **niepewność** – MoE przez specjalizację, Deep Ensembles przez różnorodność inicjalizacji.

Podsumowanie

Główne wnioski:

- **Bagging (RF):** redukuje wariancję przez dekorelację drzew (bootstrap + random subspace); łatwy w paralelizacji; nie przeuczy się
- **AdaBoost:** iteracyjna zmiana wag trudnych próbek; wrażliwy na szum i outliery; historycznie przełomowy
- **Gradient Boosting:** najsilniejsza metoda przy starannym tuningu; elastyczna funkcja straty; wymaga większego nakładu obliczeniowego
- **Stacking / Voting:** proste uzupełnienie – warto znać API sklearn

Pytania?

Zalecana literatura

- Breiman (2001), *Machine Learning* – oryginalny artykuł RF
- Friedman (2001), *Ann. Stat.* – Greedy Function Approximation
- Géron (2022), *Uczenie maszynowe z użyciem Scikit-Learn*, r. 7
- Ke et al. (2017), NeurIPS – LightGBM paper
- `docs.scikit-learn.org` – Ensemble methods

Dodatkowe źródła

- Wolpert (1992) – Stacked Generalization (oryginalny artykuł)
- Chen & Guestrin (2016), KDD – XGBoost paper